

Project 1 LLM Transcript

Muhammad Suffyan Khan

2026-02-21

LLM Transcript and Citation

Tool Used: ChatGPT (OpenAI, GPT-5.2)

Purpose: Designing and validating a reproducible parsing workflow for reconstructing relational structure from a semi-structured chess tournament cross-table

Course: DATA 607 – Project 1

Date Accessed: February 2026

Citation:

OpenAI. (2026). ChatGPT (GPT-5.2) [Large language model]. <https://chat.openai.com>.

Overview of LLM Usage

I used ChatGPT to assist with:

1. Structuring the parsing strategy for multi-line player records.
2. Designing the opponent extraction logic using regular expressions.
3. Implementing functional programming patterns using `purrr::map()`.
4. Designing validation checks (manual test cases and QA assertions).

I worked iteratively, running and testing all code locally before finalizing.

[1] Conversation with ChatGPT (Parsing Strategy Design)

How should I parse a semi-structured tournament cross-table where each player spans two lines?

Assistant (ChatGPT):

The correct approach is to: - Identify line 1 records using a pattern like `^\s*\d+\s*\|` - Identify line 2 records using state abbreviations `^[A-Z]{2}\|` - Reconstruct player-level records by binding corresponding lines - Extract pair number, player name, total points, and raw round data - Extract pre-rating from the second line using a pattern like `R:\s*([0-9]+)`

This approach ensures that parsing is rule-based rather than position-based, making the workflow more robust to spacing variations.

[2] Conversation with ChatGPT (Opponent Extraction Logic)

How should I extract opponent pair numbers while excluding byes and non-games?

Assistant (ChatGPT):

Use a regular expression that captures only valid game patterns:

`(W|L|D)\s*(\d+)`

This ensures: - Only games with a result indicator are included. - Tokens like H, B, U, or X are automatically excluded. - The denominator equals the number of actual games played.

Suggested implementation:

```
extract_opponents <- function(x) {  
  opp <- str_match_all(x, "(W|L|D)\\s*(\\d+)")[[1]]  
  if (nrow(opp) == 0) return(integer(0))  
  as.integer(opp[,3])  
}
```

[3] Conversation with ChatGPT (Functional Join + Mean Calculation)

How do I map opponent IDs to pre-ratings and compute the average cleanly?

Assistant (ChatGPT):

Create a lookup table:

```
rating_lookup <- players %>% select(pair_number, pre_rating)
```

Then use purrr::map():

Suggested implementation:

```
mutate(
  opponent_pre_ratings = map(
    opponents,
    ~ rating_lookup %>%
      filter(pair_number %in% .x) %>%
      pull(pre_rating)
  ),
  avg_opp_pre_rating = map_dbl(
    opponent_pre_ratings,
    ~ if (length(.x) == 0) NA_real_ else round(mean(.x), 0)
  )
)
```

This ensures:

- Correct relational mapping.
 - Proper denominator logic.
 - Rounded integer output.
-

[4] Conversation with ChatGPT (Validation and QA Design)

How should I validate correctness before trusting results?

Assistant (ChatGPT):

Implement two levels of validation:

Manual test cases:

- One player who played all rounds.
- One player with byes or missing rounds.
- Hand-calculate opponent rating average.

Compare with computed output.

Programmatic QA checks:

```
stopifnot(nrow(players) == 64)
stopifnot(all(all_opps \>= 1 & all_opps \<= 64))
stopifnot(!any(is.na(unlist(players$opponent_pre_ratings))))
```

This enforces structural integrity and prevents silent logical errors.

Reflection on LLM Usage

ChatGPT was primarily used as a design assistant rather than a code generator. All suggestions were:

- Tested locally before integration.
- Modified when necessary.
- Verified through manual arithmetic validation.

The model provided useful structure for regular expression extraction, functional programming design, and QA logic. However, correctness depended on iterative testing within RStudio.

This workflow reinforced that large language models can accelerate development but must always be validated independently.

ChatGPT is AI and can make mistakes.